

Bachelor Thesis

im Studiengang Geoinformatik

zur Erlangung des Grades eines
Bachelor of Science (B.Sc.)

über das Thema

LLM-gestützte Generierung von Playwright-E2E-Tests aus Use Cases

Einfluss von UI-Kontext am Beispiel von WebGIS-Anwendungen

von

Nils Große Beikel

Betreuer (Universität): Dr. Christian Knoth
Betreuer (Unternehmen): Dr. Thore Fechner
Matrikelnummer : 546548
Abgabedatum : 31. August 2026

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung und Erkenntnisinteresse	1
1.3 Aufbau der Arbeit	1
2 Grundlagen	2
2.1 Large Language Models	2
2.1.1 Funktionsprinzip und Kontext als Qualitätsfaktor	2
2.1.2 Einsatz im Software Engineering	3
2.2 E2E-Testautomatisierung mit Playwright	4
2.2.1 Teststufen und Einordnung von E2E-Tests	4
2.2.2 Playwright: Selektoren, Assertions, asynchrone UIs	5
2.3 Open Pioneer Trails	6
2.3.1 Framework-Überblick und Komponentenmodell	6
2.3.2 data-testid-Konventionen und Testbarkeit	8
2.4 WebGIS-Anwendungen und Testkomplexität	8
3 Stand der Forschung	10
3.1 LLM-gestützte Testgenerierung	10
3.2 Forschungslücke und Einordnung dieser Arbeit	10
4 Methodik und Forschungskonzept	11
4.1 Überblick und Forschungsdesign	11
4.2 Use Cases: Definition und JSON-Format	11
4.3 Versuchsaufbau: Die vier Kontextstufen	11
4.3.1 Stufe 1 - Baseline (kein Kontext)	11
4.3.2 Stufe 2 - Accessibility Snapshot via MCP	11
4.3.3 Stufe 3 - Automatisch generierte UI-Map	11
4.3.4 Stufe 4 - Manuell erstellte UI-Map	11
4.4 LLM-Konfiguration (Modell, Temperatur, Prompt-Struktur)	11
4.5 Bewertungskriterien	11

5 Implementierung der Evaluationsumgebung	12
5.1 Demo-WebGIS-Anwendung	12
5.2 Use-Case-Sammlung	12
5.3 Manuell erstellte Referenztests	12
6 Durchführung und Evaluation	14
6.1 LLM-gestützter Generierungsworkflow	14
6.2 Ergebnisse je Kontextstufe	14
6.3 Vergleich der Kontextstufen	14
6.4 Vergleich mit manuell erstellten Tests	14
6.5 GIS-spezifische Besonderheiten	14
7 Diskussion	15
7.1 Interpretation der Ergebnisse	15
7.2 Limitationen	15
7.3 Handlungsempfehlungen	15
8 Fazit und Ausblick	15
Anhang	16
Literaturverzeichnis	17

Abbildungsverzeichnis

Abbildung 1: Testpyramide nach Mike Cohn	5
Abbildung 2: Open Pioneer Trails Paketökosystem	7

Tabellenverzeichnis

Tabelle 1: Übersicht der Use Cases mit Komplexität und getesteter Kernfunktion . .	13
--	----

Abkürzungsverzeichnis

API	Application Programming Interface
ARIA	Accessible Rich Internet Applications
CSS	Cascading Style Sheets
DOM	Document Object Model
E2E	End-to-End
GIS	Geoinformationssystem
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
PNPM	Performant Node Package Manager
UI	User Interface
WMS	Web Map Service

1 Einleitung

1.1 Motivation und Problemstellung

1.2 Zielsetzung und Erkenntnisinteresse

1.3 Aufbau der Arbeit

2 Grundlagen

2.1 Large Language Models

Large Language Models (LLMs) sind maschinelle Lernmodelle, die auf sehr großen Textmengen trainiert werden, um natürliche Sprache zu verarbeiten und zu erzeugen. Das Attribut „large“ bezieht sich dabei sowohl auf den Umfang der Trainingsdaten als auch auf die Anzahl der Modellparameter, die häufig im Bereich mehrerer Milliarden liegt. Technisch beruhen sie auf künstlichen neuronalen Netzen, genauer gesagt auf der Transformer-Architektur *Hou, X. et al., 2024*. Der folgende Abschnitt erläutert zunächst das Funktionsprinzip und die Rolle des bereitgestellten Kontexts, bevor der Einsatz im Software Engineering betrachtet wird.

2.1.1 Funktionsprinzip und Kontext als Qualitätsfaktor

Moderne LLMs basieren auf der Transformer-Architektur, die von *Vaswani, A. et al., 2017* eingeführt wurde. Ihr zentraler Bestandteil ist der Self-Attention-Mechanismus. Er erlaubt es dem Modell, die Beziehungen zwischen allen Elementen einer Eingabe unabhängig von ihrer Position zu gewichten. Anders als bei früheren rekurrenten Ansätzen wird die Eingabe dadurch nicht sequentiell, sondern weitgehend parallel verarbeitet, was das Training auf sehr großen Textmengen erst praktikabel gemacht hat *ebd.* Vor der Verarbeitung wird der Text in sogenannte Tokens zerlegt. Tokens sind in der Regel keine ganzen Wörter, sondern Teilwörter oder einzelne Zeichen, auf denen das Modell dann ausschließlich arbeitet.

Die Texterzeugung erfolgt autoregressiv. Auf Basis der bereits vorliegenden Tokens berechnet das Modell für das nächste Token eine Wahrscheinlichkeitsverteilung über das gesamte Vokabular. Aus dieser Verteilung wird ein Token ausgewählt und an die Eingabe angehängt. Der Vorgang wiederholt sich, bis die Ausgabe abgeschlossen ist. Wie stark die Auswahl vom jeweils wahrscheinlichsten Token abweichen darf, steuert unter anderem der Temperatur-Parameter. Ein niedriger Wert führt zu deterministischeren, ein hoher zu variableren Ausgaben. Das ist für diese Arbeit relevant, denn nur eine über alle Versuche konstant gehaltene Konfiguration erlaubt einen aussagekräftigen Vergleich der Generierungsergebnisse *ebd.*

Alle Tokens, die das Modell für eine Generierung berücksichtigt, befinden sich im sogenannten Kontextfenster. Dieses umfasst die Eingabe (den Prompt) und die bereits erzeugte Ausgabe und ist in seiner Größe begrenzt. Wissen, das nicht in den

Modellgewichten verankert ist, also nicht während des Trainings gelernt wurde, muss dem Modell zur Laufzeit über dieses Fenster bereitgestellt werden *Brown, T. B. et al., 2020*.

Eng damit verbunden ist die Fähigkeit zum In-Context Learning. ebd. zeigten, dass große Sprachmodelle Aufgaben allein anhand von Anweisungen oder Beispielen im Prompt lösen können, ohne dass die Modellgewichte angepasst werden. Das Modell „lernt“ hierbei nicht im klassischen Sinne, sondern nutzt die im Kontext bereitgestellten Informationen direkt als Grundlage für seine Vorhersage. Welche Inhalte im Prompt stehen, bestimmt damit maßgeblich die Ausgabe.

Daraus folgt ein zentraler Punkt für diese Arbeit. Ein LLM hat kein konkretes Wissen über eine individuell entwickelte Benutzeroberfläche. Informationen über die tatsächlich vorhandenen UI-Elemente, deren Selektoren und Zustände stehen weder in den Trainingsdaten noch sind sie zur Laufzeit zugänglich, solange sie nicht explizit im Prompt bereitgestellt werden. Die Qualität des generierten Testcodes hängt damit unmittelbar von Qualität und Umfang des bereitgestellten Kontexts ab.

2.1.2 Einsatz im Software Engineering

LLMs werden zunehmend im Software Engineering eingesetzt, insbesondere zur Generierung von Quellcode. Sichtbar wird das an Werkzeugen wie GitHub Copilot, das auf Grundlage des umgebenden Quellcode-Kontexts Vorschläge zur Codevervollständigung erzeugt und direkt in die Entwicklungsumgebung integriert ist. Der systematische Literaturüberblick von *Hou, X. et al., 2024* zeigt, dass der Einsatz von LLMs im Software Engineering inzwischen breit erforscht wird und deutlich über die reine Codevervollständigung hinausreicht.

Ein Anwendungsfeld ist die automatisierte Testgenerierung. LLMs werden hier eingesetzt, um aus natürlichsprachlichen oder strukturierten Beschreibungen Unit-Tests, Akzeptanztests und End-to-End (E2E)-Tests zu erzeugen. Bestehende Ansätze beziehen sich überwiegend auf generische Webanwendungen *Celik, A., Mahmoud, Q. H., 2025; Ferreira, M. et al., 2025*. GIS-spezifische Anwendungen werden aktuell kaum berücksichtigt. Eine ausführliche Betrachtung des Forschungsstands folgt in Kapitel 3.

Diesem Potential stehen bekannte Grenzen gegenüber. Da LLMs ihre Ausgaben auf Basis von Wahrscheinlichkeiten erzeugen, können sie plausibel wirkende, faktisch aber nicht existierende Inhalte produzieren, bei der Testgenerierung etwa erfundene Selektoren oder Aufrufe nicht vorhandener Funktionen. Dieses Phänomen wird als Halluzination bezeichnet *Hou, X. et al., 2024*. Hinzu kommt das fehlende Wissen über die konkrete Anwendung und

der Umstand, dass das Modell keinen Zugriff auf den tatsächlichen Laufzeitzustand der Oberfläche hat. Gerade für E2E-Tests, die gezielt auf konkrete User Interface (UI)-Elemente zugreifen müssen, sind diese Einschränkungen besonders relevant.

Grundsätzlich lassen sich Sprachmodelle auf zwei Wegen an eine spezifische Aufgabe anpassen: durch Fine-Tuning, also das Nachtrainieren der Modellgewichte auf aufgabenspezifischen Daten, oder durch Prompting, bei dem das Verhalten allein über die Gestaltung der Eingabe gesteuert wird. Prompting beruht auf dem in Abschnitt 2.1.1 beschriebenen In-Context Learning *Brown, T. B. et al., 2020*. Diese Arbeit verfolgt durchgängig den Prompting-Ansatz, da er gegenüber Fine-Tuning sowohl einen geringeren Ressourcenaufwand erfordert als auch eine bessere Generalisierbarkeit bietet. Ein prompt-basierter Workflow lässt sich ohne erneutes Training auf andere Anwendungen und Use Cases übertragen, während ein feinjustiertes Modell weitgehend auf seinen Trainingskontext beschränkt bliebe.

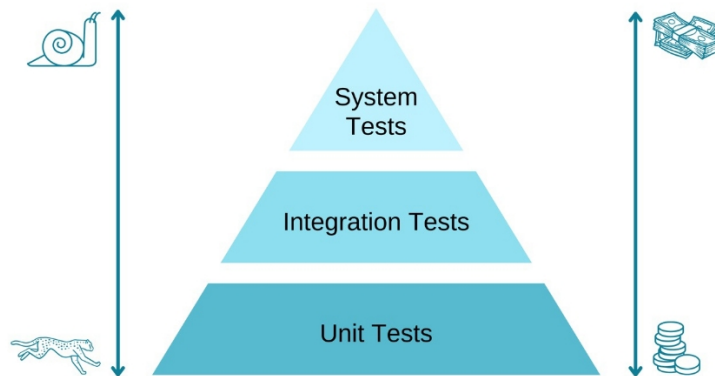
2.2 E2E-Testautomatisierung mit Playwright

Softwaretests lassen sich nach ihrem Umfang und ihrer Nähe zum Nutzerverhalten in verschiedene Stufen einteilen. Im Folgenden werden E2E-Tests in diese Stufen eingeordnet und anschließend das in dieser Arbeit verwendete Testframework Playwright vorgestellt.

2.2.1 Teststufen und Einordnung von E2E-Tests

Eine verbreitete Einteilung von Softwaretests ist die Testpyramide, die von *Poenisch, M., 2022* beschrieben wird. Sie unterscheidet drei Ebenen, wie in Abbildung 1 dargestellt. Ganz unten stehen Unit-Tests, die einzelne Funktionen oder Komponenten prüfen. Sie laufen schnell und werden in großer Anzahl geschrieben. Darüber liegen die Integrationstests, die das Zusammenspiel mehrerer Komponenten betrachten. An der Spitze stehen E2E-Tests, die die Anwendung als Ganzes aus Sicht der Nutzer prüfen.

Abbildung 1: Die Testpyramide nach Mike Cohn (Quelle: *Poenisch, M., 2022*)



Ein E2E-Test steuert dazu einen Browser und führt dieselben Schritte aus wie ein Mensch. Dazu gehören auf Schaltflächen klicken, Felder ausfüllen und durch die Oberfläche navigieren. Geprüft wird, ob am Ende das erwartete Ergebnis eintritt. Dadurch geben E2E-Tests eine gute Auskunft darüber, ob eine fachliche Anforderung tatsächlich umgesetzt ist. Allerdings sind sie langsam, aufwendig zu erstellen und vergleichsweise anfällig und können schon bei kleinen Änderungen an der UI fehlschlagen. Deshalb wird empfohlen, ihre Anzahl, entsprechend der Form der Pyramide, gering zu halten.

Für WebGIS-Anwendungen sind E2E-Tests besonders relevant. Viele Probleme zeigen sich erst im Zusammenspiel der Komponenten und Interaktionen. Dazu gehört das asynchrone Laden von Geodaten, das Rendern der Karte oder die Auswirkungen der Layersteuerung auf die Kartenanzeige. Solche Abläufe lassen sich mit Unit-Tests kaum abbilden, weil sie den vollständigen, im Browser laufenden Zustand voraussetzen.

2.2.2 Playwright: Selektoren, Assertions, asynchrone UIs

Playwright ist ein Open Source E2E-Test-Framework von Microsoft zur Automatisierung von Browsern. Es steuert die Browser-Engines Chromium, Firefox und WebKit über eine einheitliche Programmierschnittstelle und führt Tests in einem echten Browser aus. Die Tests werden in dieser Arbeit in TypeScript geschrieben, das Playwright unterstützt *Microsoft, 2026*.

Um mit der Benutzeroberfläche zu interagieren, muss ein Test die jeweiligen Elemente finden. Playwright nutzt dafür sogenannte Lokatoren. Empfohlen wird, Elemente über nutzerseitige Eigenschaften anzusprechen, etwa über ihre ARIA-Rolle oder ihren

sichtbaren Text (`getByText`, `getByRole`). Solche Selektoren sind robuster gegenüber Änderungen am Markup als Cascading Style Sheets (CSS)- oder XPath-Ausdrücke, die an die konkrete Document Object Model (DOM)-Struktur gebunden sind. Eine Alternative sind Test-IDs. Über das Attribut `data-testid` lässt sich ein Element gezielt für Tests markieren und mit `getByTestId` ansprechen, unabhängig von Darstellung und Position. Welches Attribut als Test-ID gilt, ist konfigurierbar *Microsoft, 2026*.

Ein wesentliches Merkmal von Playwright ist das automatische Warten. Vor jeder Aktion prüft das Framework, ob das Zielelement sichtbar, stabil und bedienbar ist, ansonsten wird bis zu einem Timeout gewartet. Dadurch müssen nicht für jede Interaktion manuelle Wartezeiten eingebaut werden, was eine häufige Ursache für instabile Tests beseitigt. Dasselbe Prinzip gilt für Assertions. Mit `expect` formulierte Zusicherungen wie `expect(locator).toBeVisible()` oder `toHaveText()` werden so lange wiederholt, bis die Bedingung erfüllt ist oder die Zeit abläuft. Eine Assertion prüft damit nicht einen Momentzustand, sondern einen Zustand, der sich auch verzögert einstellen darf.

Für Fälle, die das automatische Warten nicht abdeckt, stellt Playwright extra Wartemethoden bereit, etwa auf eine bestimmte Netzwerkantwort (`waitForResponse`) oder einen Ladezustand der Seite (`waitForLoadState`). Das ist gerade bei WebGIS-Anwendungen wichtig, in denen Kartenkacheln und Geodaten asynchron über das Netz geladen werden und erst danach sichtbar auftauchen. Ein Test, der zu früh prüft, würde fehlschlagen, obwohl die Anwendung korrekt arbeitet. Gerade beim asynchronen Laden von Kartenkacheln und Geodaten greift damit beides ineinander: das automatische Warten für DOM-Elemente und gezielte Wartepunkte auf Netzwerkebene.

2.3 Open Pioneer Trails

2.3.1 Framework-Überblick und Komponentenmodell

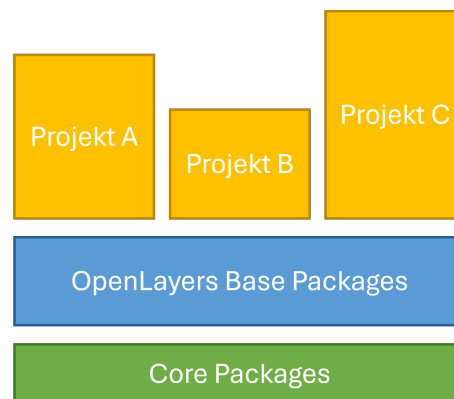
Open Pioneer Trails ist ein Open Source Framework zur Entwicklung webbasierter GeoIT-Anwendungen. Es ist als gemeinsames Projekt der con terra und 52°North im Rahmen der Open-Pioneer-Initiative entstanden und steht unter der Apache 2.0 Lizenz frei auf GitHub zur Verfügung. Trails ist ein Entwicklungsframework und kein fertiges Produkt. Anwendungen werden also nicht konfiguriert, sondern programmiert, wobei das Framework wiederverwendbare Komponenten und Vorlagen bereitstellt *52°North GmbH, Matthes Rieke Managing Director, 2026*.

Technisch besteht Trails aus modernen Webtechnologien. Die Anwendungen werden in TypeScript mit React geschrieben, das Styling basiert auf Chakra UI, als Kartenkomponente

wird OpenLayers verwendet, die Paketverwaltung läuft über Performant Node Package Manager (PNPM) und als Build-System wird Vite verwendet *Open Pioneer*, 2026.

Eine Open Pioneer Trails-Anwendung ist auf Paketen aufgebaut. Ein Paket bündelt React-Komponenten, Services, Übersetzungen und Styling. Die Anwendung selbst ist ebenfalls ein Paket und wird zu einer Web Component gebaut, dadurch kann sie einfach in bestehende Web-Anwendungen integriert werden. Aktuell werden zwei Gruppen wiederverwendbarer Pakete bereitgestellt. Die Core Packages enthalten kartenlose Basisfunktionen wie Authentifizierung, Notifier, Internationalisierung mit I18n und einen Hypertext Transfer Protocol (HTTP)-Service. Die OpenLayers Base Packages bauen darauf auf und liefern die Kartenkomponenten, darunter Layerübersicht, Kartennavigations-Werkzeuge, Legende, Selektion und Ergebnisliste, Suche, Messwerkzeug, Koordinatenanzeige und Kartendruck. Abbildung 2 gibt einen Überblick über das Paketökosystem.

Abbildung 2: Das Open Pioneer Trails Paketökosystem (Quelle: con terra interne Darstellung)



Die Anbindung der Karte erfolgt nicht direkt über OpenLayers, sondern über eine eigene Schnittstelle. Diese erweitert das Datenmodell, vereinfacht den Umgang mit dem Kartenzustand und stellt ihn als TypeScript-Schnittstelle (MapModel) bereit. Der direkte Zugriff auf die OpenLayers-Application Programming Interface (API) bleibt bei Bedarf möglich *52°North GmbH, Arne Vogt*, 2025. Für die Demo-Anwendung relevant sind unter anderem die Pakete für Karte, Layerübersicht, Basemap-Auswahl, Suche, Messen und Druck.

Trails richtet sich explizit an Entwickler und erfordert Programmierkenntnisse. Dafür lässt es Layout und Funktionalität frei gestalten und bringt typische Geoinformationssystem (GIS)-Widgets als fertige Komponenten mit. Für die Erstellung einer spezifischen Demo-WebGIS-Anwendung im Rahmen dieser Arbeit ist es damit eine passende Grundlage.

2.3.2 data-testid-Konventionen und Testbarkeit

Ein verbreitetes Mittel, um Elemente in Tests gezielt anzusprechen, ist das Hypertext Markup Language (HTML)-Attribut `data-testid`. Es wird ausschließlich für Tests vergeben und ist von Styling und DOM-Struktur unabhängig. Ein Test, der ein Element über seine Test-ID anspricht, bleibt auch dann stabil, wenn sich CSS-Klassen oder Strukturen ändern. Playwright kann dann wie in Abschnitt 2.2.2 beschrieben mit `getByTestId` darauf zugreifen.

Trails unterstützt diese Konvention standardmäßig. Alle öffentlichen Komponenten teilen sich die gemeinsame Schnittstelle `CommonComponentProps`, die optional eine `data-testid` entgegennimmt und an das DOM weiterreicht. Bei allen anderen Elementen von Chakra UI werden nur Chakra-bezogene Style-Props herausgefiltert und alles andere ans DOM übergeben, darunter auch die Test-ID *Chakra Systems*, 2026. Den Wert gibt dabei die Entwicklung vor, einen automatischen Standardwert vergibt das Framework nicht. Für gut testbare Anwendungen müssen die `data-testid`-Attribute daher bewusst und sinnvoll im Anwendungscode gesetzt werden.

Neben den Test-IDs sind die Komponenten auf Barrierefreiheit ausgelegt und folgen Accessible Rich Internet Applications (ARIA)-Konventionen *52 North GmbH, Arne Vogt*, 2025. Elemente lassen sich dadurch beispielsweise über ihre Rolle oder ihren sichtbaren Textinhalt finden. Playwright kann anhand von Rolle und Bezeichnung (`getByRole/getByText`) auf diese Elemente zugreifen. Damit bietet Trails zwei robuste Wege, ein Element zu adressieren.

2.4 WebGIS-Anwendungen und Testkomplexität

WebGIS-Anwendungen stellen die Testautomatisierung vor Herausforderungen, die generische Webanwendungen in dieser Form nicht haben. Der Grund dafür ist, wie Karten und Geodaten im Browser dargestellt und geladen werden. Ein zentrales Problem ist die Darstellung der Karte selbst. OpenLayers zeichnet die Karte auf ein Canvas-Element, also als Pixelgrafik und nicht als einzelne DOM-Elemente *OpenLayers (Institution)*, 2026. Ein Locator kann zwar das Canvas-Element finden, aber nicht die Inhalte darin, z.B. ob ein bestimmter Layer gerade sichtbar ist. Was in der Karte dargestellt wird, ist also nicht direkt über das DOM zugänglich. Zu testen, ob ein Feature auf der Karte sichtbar ist, lässt sich daher nicht auf demselben Wege prüfen wie die Sichtbarkeit einer Schaltfläche.

Hinzu kommt, dass Geodaten asynchron über das Netz geladen werden. Kartenkacheln, Web Map Service (WMS)-Layer, GetFeatureInfo-Abfragen oder Anfragen des Geocoders

treffen mit unterschiedlicher Verzögerung ein. Ein Test muss abwarten, bis die Daten geladen und gerendert wurden, bevor getestet werden kann. Das in Abschnitt 2.2.2 beschriebene automatische Warten von Playwright hilft hier nur bedingt, da es sich auf die Bedienbarkeit von DOM-Elementen bezieht. Ob Kartenkacheln vollständig dargestellt werden, lässt sich darüber nicht erkennen, sodass auf andere Methoden wie das Abwarten einer Netzwerkantwort zurückgegriffen werden muss.

Die Oberfläche einer WebGIS-Anwendung ist außerdem stark vom aktuellen Zustand abhängig. Ein Layer kann sichtbar oder unsichtbar sein, die Legende bezieht sich auf sichtbare Layer und eine Informationsanzeige erscheint erst, nachdem ein Feature in der Karte angeklickt wurde. Auch hier ist das Verhalten von Interaktionen wichtig. Die Informationsanzeige bei Klick auf ein Feature kann zum Beispiel deaktiviert sein, wenn gerade eine Messfunktion zum Messen einer Linie aktiv ist. Dieselbe Interaktion kann je nach Zustand zu einem anderen Ergebnis führen, weshalb die Vorbedingungen eines Tests genau definiert sein müssen.

Außerdem sind viele Interaktionen abhängig von Koordinaten. Angenommen der Nutzer klickt auf eine Position, auf eine Markierung oder zeichnet durch mehrere Klicks eine Linie. Der Test muss dafür auf Pixelkoordinaten im Canvas klicken statt auf ein bekanntes Element. Das Ergebnis hängt dann von Kartenausschnitt, Zoomstufe und Projektion ab. Ein Klick auf eine beliebige Position ist dabei etwas anderes als das gezielte Treffen eines bestimmten Features, das an einer bekannten Stelle liegt.

3 Stand der Forschung

3.1 LLM-gestützte Testgenerierung

3.2 Forschungslücke und Einordnung dieser Arbeit

4 Methodik und Forschungskonzept

4.1 Überblick und Forschungsdesign

4.2 Use Cases: Definition und JSON-Format

4.3 Versuchsaufbau: Die vier Kontextstufen

4.3.1 Stufe 1 - Baseline (kein Kontext)

4.3.2 Stufe 2 - Accessibility Snapshot via MCP

4.3.3 Stufe 3 - Automatisch generierte UI-Map

4.3.4 Stufe 4 - Manuell erstellte UI-Map

4.4 LLM-Konfiguration (Modell, Temperatur, Prompt-Struktur)

4.5 Bewertungskriterien

5 Implementierung der Evaluationsumgebung

5.1 Demo-WebGIS-Anwendung

5.2 Use-Case-Sammlung

Insgesamt repräsentieren 10 Use-Cases typische Nutzerinteraktionen mit der Demo-WebGIS-Anwendung. Die daraus generierten Tests sollen jeweils ihren Use-Case vollständig abdecken. Um sie auch maschinell verarbeitbar zu machen und eine einheitliche Struktur zu gewährleisten, werden sie im JavaScript Object Notation (JSON)-Format erfasst. Jeder Use-Case ist dabei wie folgt aufgebaut: `id`, `title`, `description`, `preconditions[]`, `steps[]`, `expected_result[]`, `complexity`. Titel und Beschreibung geben an, welche Nutzerinteraktion der Use-Case beschreibt. Die Vorbedingungen enthalten Informationen, die vor der Ausführung erfüllt sein müssen. Die Schritte beschreiben die durchzuführenden Nutzerinteraktionen, und die erwarteten Ergebnisse definieren, welcher Zustand nach der Ausführung eintreten soll.

Zusätzlich werden die Use-Cases in drei Komplexitätsstufen eingeteilt: `easy`, `medium` und `hard`. Die Einstufung richtet sich dabei nicht nur nach der Anzahl der Schritte, sondern vor allem nach der Art und Tiefe der Interaktionen. Einfache Use-Cases testen einzelne Schaltflächen oder Sichtbarkeiten, während komplexere Use-Cases asynchrone Prozesse, Canvas-Interaktionen oder das Zusammenspiel mehrerer Komponenten erfordern. Der komplexeste Use-Case ist UC-10, da er Layer-Sichtbarkeiten, Geocoder-Suche, Kartennavigation und Wettervorhersage in einem einzigen Ablauf kombiniert.

Eine Übersicht aller Use-Cases ist in Tabelle 1 dargestellt. Da es sich hier um eine Demo-Anwendung handelt, sind die Use-Cases bewusst begrenzt. Reale WebGIS-Anwendungen können in der Praxis deutlich umfangreichere Nutzerinteraktionen erfordern.

5.3 Manuell erstellte Referenztests

Listing 1: UseCase-1: Layer-Switcher ein-/ausblenden (Playwright-Test)

```
1 import { test, expect } from "@playwright/test";
2
3 test("UC-1: show and hide the layer switcher via the toolbar button", async ({ page })
4     => {
5     await page.goto("http://localhost:5173/ba-webgis-llm-e2e/");
6
7     const map = page.getByTestId("map-container");
```

Tabelle 1: Übersicht der Use Cases mit Komplexität und getesteter Kernfunktion

ID	Titel	Komplexität	Kernfunktion
1	Layer Switcher ein-/ausblenden	easy	Panel-Sichtbarkeit
2	Hintergrundkarte wechseln	easy	Basemap-Wechsel
3	Zoom-Buttons verwenden	easy	Zoom-Steuerung
4	UV-Index-Overlay aktivieren	medium	Layer-Rendering
5	Niederschlags-Overlay + Legende	medium	Layer + Legende
6	Wettervorhersage per Kartenklick	hard	Kartenklick-Abfrage
7	UVI-Station abfragen	hard	WMS GetFeatureInfo
8	Distanz messen	hard	Messwerkzeug
9	Karte als PNG exportieren	hard	Druckfunktion
10	Layer konfigurieren, Suche + Vorhersage	hard	Geocoder + Workflow

```

7   const layerSwitcher = page.getByTestId("layer-switcher");
8   const layerSwitcherToggle = page.getByTestId("layer-switcher-toggle");
9
10  // Vorbedingung: App geladen, Karte gerendert, TOC sichtbar.
11  await expect(map).toBeVisible();
12  await page.waitForFunction(
13    () => (globalThis as { __openPioneerMap?: unknown }).__openPioneerMap != null
14  );
15  await expect(layerSwitcher).toBeVisible();
16
17  // Schritt 1: TOC ausblenden.
18  await layerSwitcherToggle.click();
19  await expect(layerSwitcher).toBeHidden();
20
21  // Schritt 2: TOC wieder einblenden.
22  await layerSwitcherToggle.click();
23  await expect(layerSwitcher).toBeVisible();
24 });

```

6 Durchführung und Evaluation

6.1 LLM-gestützter Generierungsworkflow

6.2 Ergebnisse je Kontextstufe

6.3 Vergleich der Kontextstufen

6.4 Vergleich mit manuell erstellten Tests

6.5 GIS-spezifische Besonderheiten

7 Diskussion

7.1 Interpretation der Ergebnisse

7.2 Limitationen

7.3 Handlungsempfehlungen

8 Fazit und Ausblick

Anhang

Anhang 1: Beispielanhang

Dieser Abschnitt dient nur dazu zu demonstrieren, wie ein Anhang aufgebaut sein kann.

Anhang 1.1: Weitere Gliederungsebene

Auch eine zweite Gliederungsebene ist möglich.

Anhang 2: Bilder

Auch mit Bildern. Diese tauchen nicht im Abbildungsverzeichnis auf.

Abbildung 3: Beispielbild

Name	Änderungsdatum	Typ	Größe
 abbildungen	29.08.2013 01:25	Dateiordner	
 kapitel	29.08.2013 00:55	Dateiordner	
 literatur	31.08.2013 18:17	Dateiordner	
 skripte	01.09.2013 00:10	Dateiordner	
 compile.bat	31.08.2013 20:11	Windows-Batchda...	1 KB
 thesis_main.tex	01.09.2013 00:25	LaTeX Document	5 KB

Literaturverzeichnis

- 52 North GmbH, Arne Vogt* (2025): Open Pioneer Trails: An Open-Source Framework for Map-Oriented Web Apps, en-US, o. O., 2025-11, URL: <https://blog.52north.org/2025/11/10/open-pioneer-trails/>
- 52 North GmbH, Matthes Rieke Managing Director* (2026): Conquering new frontiers in web mapping application development, en-US, o. O., 2026-06, URL: <https://52north.org/software/software-components/open-pioneer-trails/>
- Brown, Tom B., Mann, Benjamin, Ryder, Nick, Subbiah, Melanie, Kaplan, Jared, Dhariwal, Prafulla, Neelakantan, Arvind, Shyam, Pranav, Sastry, Girish, Askell, Amanda, Agarwal, Sandhini, Herbert-Voss, Ariel, Krueger, Gretchen, Henighan, Tom, Child, Rewon, Ramesh, Aditya, Ziegler, Daniel M., Wu, Jeffrey, Winter, Clemens, Hesse, Christopher, Chen, Mark, Sigler, Eric, Litwin, Mateusz, Gray, Scott, Chess, Benjamin, Clark, Jack, Berner, Christopher, McCandlish, Sam, Radford, Alec, Sutskever, Ilya, Amodei, Dario* (2020): Language Models are Few-Shot Learners, Version Number: 4, o. O., 2020, URL: <https://arxiv.org/abs/2005.14165>
- Celik, Arda, Mahmoud, Qusay H.* (2025): A Review of Large Language Models for Automated Test Case Generation, en, in: Machine Learning and Knowledge Extraction, 7 (2025), Nr. 3, S. 97
- Chakra Systems* (2026): Chakra Factory | Chakra UI, en, o. O., 2026-06, URL: <https://chakra-ui.com/docs/styling/chakra-factory>
- Ferreira, Margarida, Viegas, Luís, Faria, João Pascoal, Lima, Bruno* (2025): Acceptance Test Generation with Large Language Models: An Industrial Case Study, in: 2025 IEEE/ACM International Conference on Automation of Software Test (AST), o. O.: IEEE, 2025-04, S. 1–11
- Hou, Xinyi, Zhao, Yanjie, Liu, Yue, Yang, Zhou, Wang, Kailong, Li, Li, Luo, Xiapu, Lo, David, Grundy, John, Wang, Haoyu* (2024): Large Language Models for Software Engineering: A Systematic Literature Review, in: ACM Trans. Softw. Eng. Methodol. 33 (2024), Nr. 8, 220:1–220:79
- Microsoft* (2026): Fast and reliable end-to-end testing for modern web apps | Playwright, en, Dokumentation, o. O., 2026-06, URL: <https://playwright.dev/>
- Open Pioneer* (2026): Open Pioneer GitHub, en, o. O., 2026-06, URL: <https://github.com/open-pioneer>
- OpenLayers (Institution)* (2026): Class: CanvasImmediateRenderer (OpenLayers v10.9.0 API), o. O., 2026-06, URL: https://openlayers.org/en/latest/apidoc/module-ol_render_canvas_Immediate-CanvasImmediateRenderer.html

Poenisch, Marie (2022): Die Testpyramide, de, o. O., 2022-08, URL: <https://www.openknowledge.de/blog/die-testpyramide>

Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, Polosukhin, Illia (2017): Attention Is All You Need, Version Number: 7, o. O., 2017, URL: <https://arxiv.org/abs/1706.03762>